



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Ciberseguridad
Análisis y desarrollo de un malware APT

Trabajo fin de estudio presentado por:	Alexander Brun Guajardo David Fernández Sanz Javier José Guzmán Rubio
Tipo de trabajo:	Trabajo Fin de Máster Grupal
Director/a:	Manuel Sánchez Rubio
Fecha:	23/04/2025

Resumen

Este proyecto tiene como objetivo desarrollar un **troyano modular** con diversas funcionalidades de acceso remoto, enfocado en el análisis de seguridad y pruebas de penetración. Se ha implementado un modelo **Cliente-Servidor** que permite la captura de pulsaciones de teclado, transferencia de archivos y ejecución remota de comandos, utilizando **TCP** y **ngrok** para la comunicación.

La metodología seguida incluye la implementación de **cifrado híbrido** mediante **AES y RSA** para asegurar la confidencialidad de los datos transmitidos. El cliente genera una clave AES y la cifra con la clave pública RSA del servidor, estableciendo una comunicación segura.

Los resultados obtenidos demuestran la viabilidad de establecer una infraestructura de control remoto con cifrado robusto. Como conclusión, este trabajo resalta la importancia del cifrado en herramientas de auditoría de seguridad y la necesidad de implementar mecanismos de protección en sistemas expuestos a este tipo de amenazas.

Importante: En el caso de un TFE grupal, la extensión del TFE debe garantizar que cada uno de los integrantes del equipo ha dedicado a la elaboración y defensa del trabajo las competencias previstas en la memoria. A modo orientativo, y para garantizar la calidad y dedicación que requiere un TFE grupal, la extensión mínima ha de ser un 50% superior a lo previsto para un TFE individual. Por lo tanto, la extensión mínima en un TFE grupal es de **75 páginas**.

Palabras clave: Ciberseguridad, Amenaza Persistente Avanzada, Criptografía, Herramienta de Acceso Remoto, Hacking Ético.

Abstract

This project aims to develop a modular trojan with various remote access functionalities, focused on security analysis and penetration testing. A Client-Server model has been implemented, allowing keystroke logging, file transfer, and remote command execution, using **TCP** and **ngrok** for communication.

The methodology includes the implementation of hybrid encryption using **AES** and **RSA** to ensure the confidentiality of transmitted data. The client generates an AES key and encrypts it with the server's RSA public key, establishing a secure communication channel.

The results obtained demonstrate the feasibility of establishing a remote-control infrastructure with strong encryption. In conclusion, this work highlights the importance of encryption in security auditing tools and the need to implement protection mechanisms in systems exposed to this type of threat.

Keywords: Cybersecurity, Advance Persistent Threat, Cryptography, Remote Access Tool, Ethical Hacking.

Índice de contenidos

Contenido

1.	Introducción	1
1.1.	Motivación	1
1.2.	Planteamiento del problema	2
1.3.	Estructura del trabajo	2
2.	Estado del arte	3
2.1.	Tecnologías utilizadas	5
2.2.	Requerimientos sw y hw	9
2.2.1.	Requerimientos Software	9
2.2.2.	Requerimientos Hardware	11
2.3.	Arquitectura del sistema	13
2.4.	Flujo del programa	15
3.	Objetivos concretos y metodología de trabajo	18
3.1.	Objetivo general	19
3.2.	Objetivos específicos	20
3.3.	Metodología del trabajo	22
4.	Desarrollo específico de la contribución	26
4.1	tipo 2. Desarrollo de software	26
4.1.1.	Identificación de requisitos	26
4.1.2.	Descripción detallada de la herramienta software desarrollada	28
4.1.3.	Evaluación	35

Índice de figuras

Figura 1. <i>Ejemplo de figura realizada para nuestro trabajo.</i>	2
--	---

Índice de tablas

Tabla 1. <i>Organización del trabajo en grupo.</i>	VIII
Tabla 2. <i>Ejemplo de tabla con sus principales elementos.</i>	2

Organización del trabajo en grupo

En este apartado se detallarán las distintas partes en las que se ha dividido el trabajo entre los componentes del grupo, los objetivos perseguidos en cada una de ellas y los mecanismos de coordinación empleados. Este apartado deberá ser validado por el director para poder comenzar con el trabajo.

Partes que aborda el TFE, distribución y estructura de la memoria

Cada estudiante debe abordar elementos/partes que, individualmente, podrían suponer un TFE por sí mismas.

Complete la siguiente tabla que debe reflejar el reparto de trabajo a la hora de confeccionar la memoria del TFE.

Tabla 1. Organización del trabajo en grupo.

Organización del trabajo en grupo - Desarrollo de la memoria	
Apartado de la memoria	Responsables
Introducción	Javier, Alexander, David
Estado del arte	Javier, Alexander, David
Objetivos y metodología de trabajo	Javier, Alexander, David
Desarrollo específico de la contribución: parte individual 1	Alexander Brun Guajardo
Desarrollo específico de la contribución: parte individual 2	Javier José Guzmán Rubio
Desarrollo específico de la contribución: parte individual 3	David Fernández Sanz
Desarrollo específico de la contribución: parte 4	Javier, Alexander, David
Conclusiones y trabajo futuro	Javier, Alexander, David

Fuente: Elaboración propia.

Objetivo del TFE desde el punto de vista de la adquisición de conocimientos

Desde el punto de vista de la adquisición de conocimientos, este Trabajo de Fin de Estudios ofrece una valiosa oportunidad formativa al abordar diversas áreas interrelacionadas dentro del ámbito de la ciberseguridad, permitiendo a los estudiantes adquirir una visión integral y práctica de los desafíos actuales en este campo. La naturaleza multidisciplinar del proyecto no solo abarca aspectos teóricos fundamentales, sino que también se orienta a la aplicación práctica de dichos conocimientos en contextos reales.

En particular, para aquellos profesionales enfocados en funciones de defensa de sistemas de información —como los equipos *Blue Team*— este TFE representa una herramienta de aprendizaje especialmente relevante. El análisis detallado de amenazas, vulnerabilidades y medidas de mitigación les permite comprender con mayor profundidad las técnicas, tácticas y procedimientos utilizados por potenciales atacantes, así como desarrollar estrategias de defensa más efectivas.

Además, el estudio realizado en este trabajo, al simular escenarios de riesgo y proponer mecanismos de protección, resulta directamente aplicable al entorno profesional. Esto proporciona a los participantes una experiencia que trasciende lo académico y se convierte en una base sólida para su desempeño en roles relacionados con la protección de infraestructuras críticas, la gestión de incidentes o la implementación de controles de seguridad.

En definitiva, el TFE contribuye de forma significativa a la formación de profesionales más preparados y conscientes del panorama actual de amenazas, capacitados no solo para responder ante incidentes, sino también para anticiparse a ellos desde una perspectiva estratégica y fundamentada.

Mecanismos de coordinación empleados

Para garantizar una adecuada coordinación y un desarrollo eficaz del Trabajo de Fin de Estudios (TFE) grupal, se ha optado por seguir una metodología de trabajo ágil, concretamente el marco de trabajo *Scrum*. Esta metodología permite una gestión iterativa e incremental del proyecto, fomentando la colaboración, la adaptación continua y la mejora constante.

Dentro de este marco, se realizarán dos reuniones de seguimiento semanales con todos los integrantes del grupo. Estas reuniones tendrán como objetivo principal revisar el avance de las tareas asignadas, identificar posibles obstáculos, reajustar los objetivos si fuera necesario, y planificar las acciones para el siguiente periodo de trabajo. Este sistema de reuniones periódicas permitirá mantener una visión clara y actualizada del estado del proyecto en todo momento.

En cuanto a las herramientas de comunicación, se utilizarán dos plataformas principales. Para la comunicación diaria e instantánea entre los miembros del grupo, se empleará una aplicación de mensajería instantánea (WhatsApp), que facilitará la coordinación rápida y ágil de tareas puntuales. Asimismo, se utilizará una plataforma de voz y videollamadas (Discord) para la realización de reuniones virtuales y la discusión de temas que requieran mayor profundidad o una interacción más directa.

Estas herramientas y mecanismos permitirán una coordinación efectiva del equipo, asegurando una comunicación fluida y una correcta gestión del proyecto en todas sus fases.

1. INTRODUCCIÓN

El presente trabajo aborda el diseño, desarrollo y evaluación de una herramienta que simula el comportamiento de una amenaza persistente avanzada (APT) dentro de un entorno controlado. A diferencia de otros tipos de malware que buscan un impacto inmediato, las APT están diseñadas para permanecer ocultas durante largos periodos, recolectando información sensible o comprometiendo activos estratégicos. Su nivel de sofisticación y su impacto en sectores críticos justifican la necesidad de herramientas académicas que permitan estudiar su comportamiento sin generar un riesgo real.

La herramienta desarrollada simula las principales capacidades de un malware APT: persistencia avanzada, recolección de datos, comunicación encubierta con un servidor de comando y control (C2) y técnicas básicas de evasión. Este entorno de simulación permite analizar la eficacia de diferentes soluciones defensivas y plantea un escenario de entrenamiento para profesionales del ámbito de la ciberseguridad ofensiva y defensiva.

Este capítulo presenta la motivación del trabajo, el planteamiento del problema que se aborda y la estructura general de la memoria.

1.1. MOTIVACIÓN

Las amenazas persistentes avanzadas (APT) representan una de las formas más peligrosas y complejas de ciberataques. Han sido utilizadas por actores estatales y grupos organizados para infiltrarse en redes gubernamentales, militares y empresariales, operando de forma encubierta durante semanas o incluso meses. Comprender cómo operan estas amenazas es esencial para el desarrollo de defensas efectivas.

Este trabajo surge de la necesidad de contar con entornos simulados donde se pueda observar y estudiar el comportamiento de una APT sin riesgo real. La creación de un simulador controlado no solo permite formar a analistas en ciberseguridad, sino que también posibilita la validación de tecnologías defensivas bajo condiciones realistas.

1.2. PLANTEAMIENTO DEL PROBLEMA

Las herramientas tradicionales de detección de malware suelen fallar frente a amenazas APT, debido a su enfoque evasivo y su comportamiento en apariencia benigno. Los sistemas actuales carecen de entornos de entrenamiento que simulen la complejidad de una APT sin comprometer la seguridad del entorno productivo.

El presente trabajo propone como solución el desarrollo de una herramienta académica que simule un malware tipo APT. Dicha herramienta incluirá funcionalidades típicas de estas amenazas: ejecución oculta, persistencia, recopilación de información y comunicación remota cifrada. Su diseño se realizará siguiendo los principios de seguridad, evitando cualquier capacidad destructiva o fuera de control.

1.3. ESTRUCTURA DEL TRABAJO

Este documento se estructura en cinco capítulos principales. En el capítulo 2 se revisa el estado del arte sobre amenazas APT y las tecnologías de defensa actuales. El capítulo 3 presenta los objetivos generales y específicos del trabajo, junto con la metodología adoptada. El capítulo 4 describe el desarrollo de la herramienta, incluyendo sus requisitos, arquitectura, implementación y evaluación. Finalmente, el capítulo 5 expone las conclusiones del trabajo y las posibles líneas de investigación futura.

2. ESTADO DEL ARTE

Las amenazas persistentes avanzadas (APT) se caracterizan por su complejidad técnica, objetivos específicos y duración prolongada en sistemas comprometidos. Estas amenazas suelen estar patrocinadas por estados o grupos organizados, y se dirigen contra infraestructuras críticas, instituciones gubernamentales, sectores industriales y empresas tecnológicas con el fin de robar información sensible o causar daños estratégicos.

La evolución de las APT ha ido de la mano con el desarrollo de nuevas técnicas de evasión, cifrado, comunicación encubierta y movimiento lateral. Ejemplos emblemáticos como Stuxnet, APT28, APT29, Equation Group o Lazarus han demostrado el impacto que estas amenazas pueden tener a nivel geopolítico y económico.

Desde el punto de vista defensivo, los mecanismos tradicionales basados en firmas se han visto insuficientes. Por ello, se han desarrollado tecnologías como los sistemas EDR (Endpoint Detection and Response), SIEM (Security Information and Event Management), reglas YARA, y soluciones basadas en inteligencia artificial para detectar comportamientos anómalos. Sin embargo, muchas de estas tecnologías carecen de entornos de prueba que simulen amenazas APT de forma segura.

La mayoría de los simuladores disponibles se centran en malware común o ataques simples. Herramientas como Metasploit, Cobalt Strike o Caldera permiten simular fases de ataques avanzados, pero suelen requerir conocimientos técnicos elevados o no están diseñadas con un enfoque didáctico/académico. Este trabajo busca llenar ese vacío, proporcionando una herramienta educativa que combine realismo técnico y control total del entorno.

A lo largo del tiempo se han documentado varios grupos APT con diferentes características. A continuación, se describen los más relevantes y las funcionalidades que comparten con las que se plantean en este proyecto:

- **Stuxnet (sabotaje industrial encubierto).** Este fue uno de los primeros ejemplos ampliamente conocidos de un ataque APT que no solo afectaba digitalmente, sino que también tuvo consecuencias físicas. Fue descubierto en 2010 y su objetivo principal era sabotear centrifugadoras utilizadas en el programa nuclear iraní. La sofisticación de su diseño y la precisión de sus objetivos sugieren que fue desarrollado por actores estatales con altos recursos técnicos. A diferencia de otros malware más “genéricos”, Stuxnet estaba programado para actuar únicamente bajo condiciones muy específicas, evitando así dañar sistemas fuera de su objetivo principal.
 - Usaba múltiples vulnerabilidades de día cero (0-day) para propagarse sin ser detectado.
 - Se introducía mediante memorias USB, lo que evitaba depender de internet.
 - Permitía ejecutar comandos directamente sobre sistemas industriales SCADA.

- Intervenia el funcionamiento de los controladores PLC, modificando parámetros físicos sin levantar alertas inmediatas.
- Considerado el primer “ciberarma” real por su impacto físico directo y la precisión de su objetivo.
- **APT28 (campañas militares y ciberespionaje).** También conocido como *Fancy Bear*, este grupo ha sido vinculado al servicio de inteligencia militar ruso (GRU). Está activo desde al menos 2007 y ha estado detrás de numerosas campañas de ciberespionaje, especialmente contra la OTAN, gobiernos europeos y elecciones en distintos países. Utiliza herramientas que les permiten mantener acceso remoto a los sistemas infectados y obtener información sensible de forma continua.
 - Desarrollaron malware como **X-Agent**, capaz de:
 - Ejecutar comandos de forma remota.
 - Registrar pulsaciones de teclado (keylogging).
 - Tomar capturas de pantalla.
 - Exfiltrar archivos y documentos personales.
 - Emplean infraestructuras de comando y control (C2) cifradas.
 - Han utilizado correos electrónicos de spear-phishing muy dirigidos para el acceso inicial.
 - Se adaptan rápidamente a nuevas defensas y actualizan sus herramientas constantemente.
- **APT29 (operaciones encubiertas a largo plazo).** Conocido también como *Cozy Bear*, este grupo está relacionado con el Servicio de Inteligencia Exterior ruso (SVR). Su enfoque es aún más discreto y prolongado que el de APT28, con campañas diseñadas para durar años dentro de las redes comprometidas sin ser detectados. Ha atacado ministerios, embajadas, organismos sanitarios y empresas tecnológicas.
 - Usa malware modular que puede personalizarse para cada ataque.
 - Sus herramientas permiten:
 - Ejecutar scripts remotamente sin dejar rastros evidentes.
 - Registrar todo lo que se escribe en el teclado (keyloggers).
 - Obtener contraseñas y credenciales almacenadas.
 - Utilizan técnicas avanzadas de evasión, como cifrado, suplantación de procesos legítimos y uso de servidores proxy.
 - Se cree que fueron responsables del ataque al Comité Nacional Demócrata en 2016.
- **Lazarus Group (ciberdelincuencia con fines estatales).** Este grupo, vinculado a Corea del Norte, combina actividades de espionaje con robos financieros a gran escala. Se ha vuelto conocido tanto por el ataque a Sony Pictures en 2014 como por sus robos a bancos y casas de cambio de criptomonedas. Su estrategia mezcla técnicas de APT con métodos más agresivos y visibles.
 - Utiliza puertas traseras (*backdoors*) personalizadas para el control remoto total del sistema.

- Integra keyloggers y mecanismos para robar credenciales bancarias y datos financieros.
 - Emplea servicios legítimos como Dropbox o GitHub como canales de comunicación, dificultando su detección.
 - Han ejecutado ataques de ransomware, como *WannaCry*, que afectaron a miles de sistemas en 2017.
- **Equation Group (ciberarmas altamente sofisticadas).** Considerado uno de los grupos APT más avanzados técnicamente, ha sido vinculado a la NSA (Agencia de Seguridad Nacional de EE.UU.) por la comunidad de expertos en ciberseguridad. Sus herramientas fueron reveladas por el grupo Shadow Brokers, lo que permitió conocer su nivel de complejidad y alcance.
 - Capaces de modificar el *firmware* de discos duros, permitiendo una persistencia casi imposible de eliminar.
 - Sus plataformas de control remoto usan protocolos personalizados y cifrados para evitar la detección.
 - Disponen de herramientas que permiten explotar múltiples sistemas operativos, incluyendo versiones antiguas y sin soporte.
 - Utilizan canales de exfiltración encubiertos que imitan tráfico normal, lo que complica su identificación en redes grandes.
 - Se considera que estaban detrás de la herramienta **DoublePulsar**, que se usó como base para otros ataques, incluido *WannaCry*.

En resumen, los grupos APT comparten una serie de funcionalidades comunes —como la ejecución remota de comandos, el registro de actividad mediante keyloggers, y el uso de canales encubiertos— que también se implementarán en el proyecto de este trabajo. Estas amenazas representan un reto enorme para los equipos de ciberseguridad por su nivel técnico, su capacidad de ocultación y su enfoque estratégico. Analizar su funcionamiento es esencial para desarrollar entornos de simulación y formación donde se puedan estudiar de forma segura sus tácticas, técnicas y procedimientos.

2.1. TECNOLOGÍAS UTILIZADAS

El desarrollo de este proyecto, centrado en la simulación de una Amenaza Persistente Avanzada (APT), ha requerido la selección de tecnologías que permitan replicar de forma realista el comportamiento de una campaña maliciosa. Las herramientas elegidas han sido C# como lenguaje principal para la construcción del cliente y del servidor, Ngrok como sistema de transporte para facilitar la comunicación entre ambas partes, y el almacenamiento en archivos CSV y texto plano como método sencillo y directo para guardar la información

recolectada. Esta elección no es casual: responde a la necesidad de mantener el proyecto manejable y funcional sin renunciar a una lógica y estructura cercanas a escenarios reales de ciberataques.

Cada una de estas tecnologías cumple un rol concreto dentro de la arquitectura del proyecto y ha sido escogida por sus ventajas técnicas, su facilidad de uso y su capacidad de integración. A continuación, se detalla el funcionamiento de cada tecnología, sus beneficios y limitaciones, y cómo interactúan entre sí para dar forma a una infraestructura funcional desde el punto de vista del atacante.

C#

C# es un lenguaje de programación moderno y versátil, ampliamente utilizado en entornos Windows y muy popular dentro del desarrollo empresarial y de herramientas de escritorio. Está fuertemente tipado, es orientado a objetos y permite acceder con relativa facilidad a funcionalidades avanzadas del sistema operativo, lo que lo hace ideal para el desarrollo de software que requiere controlar procesos, gestionar archivos, comunicarse por red o interactuar con el hardware del equipo. En este proyecto, C# se ha utilizado tanto para el desarrollo del cliente, que se ejecuta en la máquina de la víctima, como para el servidor, que permanece en espera en la máquina del atacante.

Una de las razones principales para utilizar C# es su profundo nivel de integración con el sistema operativo Windows, permitiendo simular funciones comunes en ataques reales, como la ejecución de comandos, el keylogging, o el acceso a archivos y directorios del sistema. Gracias a su amplia librería estándar y a las funciones del framework .NET, ha sido posible desarrollar estas funcionalidades de forma directa, sin necesidad de incorporar librerías de terceros o soluciones complejas. Asimismo, la capacidad de estructurar el código en clases y métodos ha facilitado la organización del proyecto, separando la lógica de red, la gestión de archivos y el procesamiento de comandos.

Otro punto fuerte de C# es la disponibilidad de herramientas de desarrollo como Visual Studio, que incluye depuración en tiempo real, gestión de paquetes NuGet y una interfaz muy cómoda para trabajar. Esto ha permitido avanzar de forma rápida y segura durante la implementación. Como lenguaje, C# también ofrece un buen rendimiento, estabilidad y legibilidad del código, lo que reduce el riesgo de errores y facilita futuras ampliaciones.

Ngrok

Ngrok es una herramienta muy útil para exponer servicios locales a través de Internet. En esencia, actúa como un túnel que conecta un puerto local de nuestra máquina con una URL pública accesible desde cualquier parte del mundo. En este proyecto, Ngrok se ha utilizado para exponer el servidor desarrollado en C#, que normalmente solo estaría disponible en la red local. Gracias a esta herramienta, se genera una dirección pública (del tipo `https://xxxx.ngrok.io`) que el cliente puede utilizar para comunicarse con el servidor, independientemente de la ubicación física o red de cada dispositivo.

El uso de Ngrok es especialmente interesante en un contexto como este, donde se simula un ataque en el que el atacante no necesariamente tiene acceso a redes corporativas ni puede abrir puertos en un router. Ngrok resuelve ese problema de manera elegante y sencilla, permitiendo conexiones remotas incluso desde entornos restringidos. Además, toda la comunicación que atraviesa el túnel está cifrada de extremo a extremo, lo que añade una capa de protección contra inspecciones de red o mecanismos de defensa como IDS o firewalls que analizan tráfico en texto plano.

Otra ventaja destacable es que Ngrok permite ver en tiempo real las peticiones que llegan al servidor y su contenido, lo que ha sido de gran ayuda durante la fase de desarrollo y pruebas. Sin embargo, también hay que tener en cuenta ciertas limitaciones: en su versión gratuita, los túneles caducan cada cierto tiempo y la URL cambia, lo cual obliga a actualizar el cliente si se quiere mantener una conexión continua. Aun así, para fines de simulación, pruebas y prácticas educativas, su facilidad de uso y configuración lo convierten en una herramienta excelente.

CSV Y Texto Plano

En este proyecto, se ha optado por un sistema de almacenamiento basado en archivos CSV y ficheros de texto plano para guardar toda la información que el servidor recibe desde los clientes. Esta decisión responde principalmente al objetivo de mantener el sistema lo más simple posible, evitando depender de bases de datos externas o de soluciones que requieran configuración y mantenimiento adicional. Por cada cliente que se conecta, el servidor genera uno o varios archivos que contienen los datos enviados por el cliente: resultados de comandos, pulsaciones de teclado, e incluso logs de comportamiento.

El uso de archivos CSV (Comma-Separated Values) permite organizar los datos en forma de tabla, con filas y columnas, lo que facilita su posterior análisis con herramientas como Excel o

Python. Por ejemplo, se puede almacenar una fila por cada comando ejecutado, indicando la hora, el comando enviado y el resultado recibido. Esto da una estructura clara y reutilizable. Por otro lado, los archivos de texto plano son útiles para guardar secuencias largas de texto sin necesidad de estructurarlos formalmente, como logs del sistema, contenido del portapapeles o incluso capturas de pantalla en formato codificado.

Este método de almacenamiento tiene la ventaja de ser completamente local, rápido y accesible. Cualquier desarrollador o usuario puede abrir estos ficheros sin requerir software especializado. Además, permite mantener un archivo independiente por cada cliente, simulando cómo un atacante llevaría un registro personalizado de cada víctima. No obstante, hay que reconocer también sus limitaciones: no ofrece mecanismos de seguridad ni cifrado, no escala bien si se manejan cientos de víctimas, y puede volverse difícil de organizar si se generan muchos archivos.

Ventajas y desventajas de las tecnologías utilizadas

El conjunto de tecnologías utilizadas ha permitido construir un entorno funcional, didáctico y cercano a situaciones reales. C# destaca por su capacidad de acceso al sistema y su facilidad para desarrollar tanto cliente como servidor con un único lenguaje. Esto facilita la implementación y reduce los errores de compatibilidad. Sin embargo, tiene el inconveniente de estar muy ligado al ecosistema Windows, lo que impide una portabilidad directa a otros sistemas operativos. Para ampliar el alcance del proyecto a dispositivos móviles o entornos Linux, sería necesario adaptar o reescribir el código.

Ngrok, por su parte, permite establecer fácilmente una comunicación entre cliente y servidor sin preocuparse por la red o la ubicación geográfica. Su principal ventaja es la rapidez con la que se puede publicar un servicio local y obtener una dirección pública funcional. No obstante, depende de un servicio externo, y la versión gratuita presenta ciertas limitaciones (duración de túneles, cambio de URL, velocidad). Para un atacante real, esto podría implicar la pérdida de persistencia si el túnel se cierra y los clientes no pueden volver a conectar.

En cuanto al almacenamiento en archivos, su simplicidad es su mayor virtud. No requiere configuraciones complicadas, ni servidores, ni acceso remoto, y permite conservar fácilmente la información recogida. Sin embargo, desde un punto de vista profesional, no es una solución escalable ni segura. Ante un número elevado de víctimas o ante la necesidad de análisis

avanzado, sería recomendable migrar a una base de datos más robusta y segura, como SQLite o PostgreSQL.

Integración cliente, servidor y ngrok

La integración entre las distintas piezas del proyecto ha sido diseñada de forma que todo el sistema funcione con fluidez y naturalidad. El servidor, escrito en C#, se ejecuta en la máquina del atacante y queda a la espera de conexiones entrantes en un puerto concreto. Ngrok se encarga de publicar ese puerto en Internet, creando una dirección accesible desde cualquier lugar. Por su parte, el cliente (también desarrollado en C#) consulta esa URL, se conecta, recibe instrucciones y ejecuta comandos en el sistema de la víctima. A su vez, el cliente envía los resultados de vuelta al servidor, que los almacena de forma local.

Esta arquitectura permite simular una comunicación real entre una máquina infectada y un centro de control remoto (C2), como sucede en los ataques avanzados. A diferencia de sistemas más complejos que usan infraestructura propia, esta solución permite replicar ese mismo comportamiento con recursos mínimos. Además, el uso de Ngrok permite que el atacante no tenga que preocuparse por cuestiones de red como la apertura de puertos o la obtención de una IP pública fija.

El flujo de trabajo es simple pero efectivo: Ngrok expone el servidor, el cliente se conecta, se envía una orden, se ejecuta y se devuelven los resultados. Cada conjunto de datos se guarda en un archivo dedicado, simulando cómo un atacante podría mantener organizado su “panel de víctimas”. Esta estructura modular, directa y sin dependencias externas permite no solo simular un ataque real, sino también observar y analizar en detalle cada paso del proceso.

2.2.REQUERIMIENTOS SW Y HW

Para garantizar el correcto funcionamiento del sistema desarrollado, tanto en su parte cliente como en su componente servidor, es fundamental contar con unos requisitos básicos a nivel de software y hardware. Esta sección tiene como objetivo detallar todos esos elementos necesarios, explicando por qué se ha optado por ciertas tecnologías y qué configuraciones han sido las más adecuadas durante las fases de desarrollo, pruebas y ejecución.

2.2.1. Requerimientos Software

El sistema ha sido diseñado y probado para funcionar sobre sistemas operativos Windows, ya que este entorno es compatible con todas las tecnologías utilizadas, especialmente con .NET

y Visual Studio. Se recomienda utilizar Windows 10 o Windows 11, ambos en su versión de 64 bits (x64), ya que estas versiones ofrecen mayor estabilidad, mejor soporte para hardware moderno y actualizaciones de seguridad periódicas. También es totalmente funcional en sistemas como Windows Server 2016 o versiones superiores, que pueden ser especialmente útiles si se desea desplegar el servidor en un entorno corporativo o con necesidades más avanzadas en términos de red y control de usuarios. Tener un sistema operativo actualizado es esencial para evitar incompatibilidades y errores inesperados durante la ejecución.

Entorno de Desarrollo

Todo el desarrollo del proyecto, incluyendo tanto el servidor como el cliente, se ha llevado a cabo utilizando Visual Studio 2022. Este entorno de desarrollo integrado ha sido elegido por su completa integración con el lenguaje C# y el framework .NET, además de por su facilidad de uso y las herramientas avanzadas que ofrece. Visual Studio proporciona funcionalidades muy útiles como el autocompletado inteligente (IntelliSense), depuración en tiempo real, control de versiones mediante Git y una gestión eficiente de paquetes y dependencias con NuGet. Además, su entorno gráfico facilita el diseño, la organización y el despliegue de los distintos proyectos que componen el sistema. Usar un entorno robusto y profesional como Visual Studio permite trabajar de forma más cómoda, ordenada y segura.

Framework de Ejecución

Para poder ejecutar correctamente las aplicaciones tanto en el lado del cliente como del servidor, es necesario contar con el .NET Framework 4.7.2 o una versión superior. Este framework es una plataforma de desarrollo proporcionada por Microsoft que incluye todas las librerías necesarias para trabajar con programación orientada a objetos, sockets TCP, manejo de procesos, acceso a archivos, y otras funcionalidades esenciales que se utilizan en este proyecto. Además, .NET Framework se encuentra preinstalado en muchas versiones de Windows o puede descargarse de forma gratuita desde el sitio oficial de Microsoft. Su uso facilita el desarrollo de aplicaciones seguras, estables y fáciles de mantener.

Conectividad y Comunicación Remota

Una parte esencial del sistema es la comunicación remota entre el servidor y los distintos clientes, por lo que contar con una buena conectividad es clave. Para que esta comunicación funcione correctamente, es necesario que los dispositivos estén conectados a la misma red

local o, en caso contrario, que dispongan de acceso a Internet. Si se desea conectar clientes que están fuera de la red local, es imprescindible realizar una redirección de puertos en el router hacia la IP del servidor, lo cual puede ser complejo o incluso inviable en algunos entornos. Para evitar esta limitación, se ha optado por integrar el servicio de Ngrok, una herramienta que permite crear túneles seguros a través de Internet, facilitando así la conexión entre cliente y servidor sin necesidad de modificar la configuración del router. Esta solución ha demostrado ser sencilla, práctica y muy eficaz durante las pruebas.

Recomendaciones de Seguridad y Estabilidad

Durante el uso del sistema, se recomienda añadir la carpeta donde se aloja el ejecutable del servidor a la lista de exclusiones del antivirus o del firewall, ya que algunos de sus comportamientos —como la escucha de puertos TCP o la ejecución remota de comandos— pueden ser identificados erróneamente como maliciosos. Esto puede dar lugar a bloqueos o eliminaciones por parte del software de seguridad, afectando negativamente al funcionamiento del sistema. Asimismo, es importante asegurarse de que el equipo que actúa como servidor no entre en suspensión ni en hibernación, ya que esto interrumpiría la escucha del socket y, por tanto, impediría la conexión de nuevos clientes.

2.2.2. Requerimientos Hardware

Durante la fase de pruebas del sistema, se utilizaron tres equipos distintos con configuraciones similares. Todos ellos superan con creces los requisitos mínimos para la ejecución del sistema, lo que permitió comprobar su rendimiento en condiciones reales y simular situaciones con múltiples clientes conectados. A continuación, se presentan las características de cada equipo utilizado en forma de tabla, seguidas de una breve descripción.

Característica	Especificación
Marca	MSI
Modelo	GL65 Leopard
Procesador	Intel Core i7-10750H

Característica	Especificación
Memoria RAM	32 GB
GPU	NVIDIA GeForce RTX 2060
Pantalla	15.6 pulgadas, Full HD, 120Hz
Almacenamiento	1 TB SSD
Sistema Operativo	Windows 10

Especificaciones del primer equipo

Característica	Especificación
Placa Base	ASUS
Procesador	Intel Core i7-10750H
Memoria RAM	32 GB
GPU	NVIDIA GeForce RTX 2060
Almacenamiento	Samsung 970 Evo Plus 1 TB SSD
Monitor	Curvo, 27 pulgadas, 165Hz
Sistema Operativo	Windows 10

Especificaciones del segundo equipo

Característica	Especificación
Marca	HP
Modelo	Omen 15
Procesador	AMD Ryzen 7 4800H
Memoria RAM	16 GB
GPU	NVIDIA GeForce GTX 1660 Ti
Pantalla	15.6 pulgadas, Full HD, 144Hz
Almacenamiento	512 GB SSD
Sistema Operativo	Windows 11

Especificaciones del tercer equipo

2.3.ARQUITECTURA DEL SISTEMA

Para este proyecto, se ha optado por una arquitectura basada en tres capas bien diferenciadas: la capa de presentación, la capa de transporte y la capa de procesamiento de datos. Esta organización estructural tiene como objetivo principal mejorar la modularidad del sistema, permitiendo una mayor claridad en la separación de funciones, facilitando futuras ampliaciones y reduciendo el riesgo de errores al hacer cambios. La elección de esta arquitectura también responde a la necesidad de manejar de manera eficiente la comunicación entre el cliente y el servidor, especialmente en escenarios donde se establece una conexión remota a través de internet utilizando herramientas como Ngrok.

La capa de presentación es la encargada de ofrecer la interfaz y funcionalidades visibles al usuario final, en este caso el operador del servidor. Esta capa se implementa mediante una aplicación de consola desarrollada en C#, que permite al operador gestionar y visualizar de forma clara las acciones realizadas por los clientes conectados. A través de esta interfaz se pueden ejecutar comandos en los equipos comprometidos, recibir datos recolectados por el

keylogger, capturar pantallas del sistema remoto y gestionar la lista de clientes conectados. La aplicación ha sido diseñada para ser ligera, intuitiva y de fácil manejo, evitando así la sobrecarga visual o técnica para el usuario del sistema.

Toda la lógica de interacción con el sistema pasa inicialmente por esta capa, desde donde se disparan los comandos que posteriormente son transmitidos al servidor. Aunque no se ha optado por una interfaz gráfica avanzada, la simplicidad de la consola es suficiente para cumplir los objetivos del proyecto, manteniendo una respuesta rápida y clara frente a múltiples conexiones.

La capa de transporte se encarga de facilitar la comunicación entre el servidor y los clientes. En este caso, se ha utilizado el protocolo TCP como base para establecer una comunicación estable y persistente entre los distintos elementos del sistema. Este tipo de conexión permite mantener un canal abierto a través del cual el servidor puede recibir información o enviar instrucciones en tiempo real. Para facilitar la conexión entre el cliente (que puede estar detrás de un firewall o una red NAT) y el servidor, se ha integrado el uso de Ngrok, una herramienta que permite establecer túneles seguros a través de Internet.

Gracias a Ngrok, el servidor puede estar accesible de forma remota sin necesidad de configurar manualmente el reenvío de puertos en el router o realizar modificaciones complejas en la red del cliente. Esta herramienta redirige las peticiones que recibe en una dirección pública hacia el puerto local donde se encuentra escuchando el servidor, estableciendo así un canal de comunicación seguro, rápido y fiable. Este enfoque es especialmente útil para realizar pruebas desde diferentes ubicaciones, permitiendo simular ataques reales en distintos entornos de red, incluso si los equipos no se encuentran en la misma red local.

La última capa, correspondiente al procesamiento de datos, se encuentra principalmente centralizada en el servidor. Aquí es donde se recibe, gestiona y almacena toda la información procedente de los clientes conectados. Entre los datos procesados se incluyen registros de pulsaciones de teclas (keylogger), capturas de pantalla, nombres de procesos activos, respuestas a comandos específicos y cualquier otro tipo de información que los clientes estén programados para enviar.

Además del almacenamiento y la visualización, esta capa también se encarga de aplicar la lógica de negocio, es decir, interpretar los comandos recibidos por parte del operador y traducirlos en instrucciones específicas que se envían a los clientes. Asimismo, se gestiona la respuesta de estos clientes y se organiza de forma adecuada para su presentación en la consola. Por cuestiones de simplicidad y eficiencia, no se ha utilizado una base de datos persistente, sino que los datos se almacenan temporalmente en la memoria o en archivos locales del sistema, lo que es suficiente para el objetivo experimental y demostrativo del proyecto.

En conjunto, esta arquitectura de tres capas permite una clara separación entre la lógica de presentación, la comunicación de red y el procesamiento de datos. Este enfoque no solo facilita el desarrollo y mantenimiento del sistema, sino que también proporciona una base sólida para futuras ampliaciones, como la inclusión de una interfaz gráfica, la integración de una base de datos relacional o el despliegue del servidor en un entorno cloud. Además, el uso de Ngrok en la capa de transporte refuerza el enfoque práctico del sistema, al permitir que el servidor sea accesible desde cualquier parte del mundo sin complicaciones de configuración de red, haciendo que la arquitectura sea tanto funcional como adaptable.

2.4.FLUJO DEL PROGRAMA

Para comprender mejor el funcionamiento de la solución desarrollada, en esta sección se describen los diagramas de flujo que representan el proceso de conexión y operación entre el servidor (controlador) y el cliente (víctima). Estos diagramas tienen como objetivo ofrecer una representación visual clara y estructurada de cómo se establece la comunicación entre ambos extremos, así como de las acciones que se pueden llevar a cabo una vez establecida dicha conexión.

El funcionamiento de la herramienta desarrollada se inicia en el momento en que el cliente, que representa al equipo víctima, ejecuta la aplicación. En ese instante, se establece automáticamente una solicitud de conexión hacia el servidor, el cual permanece constantemente a la espera de nuevas conexiones gracias al componente TcpListener. Esta primera fase corresponde al establecimiento del canal TCP, que es el punto de partida para toda la comunicación entre ambos extremos.

Una vez que el servidor acepta la solicitud del cliente, se inicia el intercambio criptográfico que permite asegurar la confidencialidad de toda la información que se transmitirá durante la sesión. El servidor genera, de manera dinámica, un nuevo par de claves RSA: una pública y otra privada. Solo la clave pública es enviada al cliente. Esta clave tiene como propósito permitir al cliente cifrar información sensible de manera que únicamente el servidor, poseedor de la clave privada, pueda descifrarla.

Con la clave pública ya recibida, el cliente procede a generar una clave AES aleatoria, la cual será utilizada como clave simétrica para cifrar la comunicación de forma rápida y segura durante el resto de la sesión. Esta clave AES es cifrada utilizando la clave pública RSA que el cliente recibió del servidor, y posteriormente es enviada de vuelta al servidor. Gracias a este proceso, solo el servidor podrá descifrar la clave AES con su clave privada, completando así el establecimiento de un canal seguro basado en cifrado híbrido (asimétrico para el intercambio inicial y simétrico para la comunicación continua).

A partir de este momento, ambas partes —cliente y servidor— comparten la misma clave AES, lo que permite que todos los datos intercambiados estén protegidos mediante un cifrado robusto de tipo AES-256. Esta capa de cifrado garantiza que la información enviada, como comandos, registros de teclado (keylogs), información del sistema o archivos, no pueda ser interceptada ni comprendida por terceros.

Una vez establecida esta comunicación cifrada, el servidor adquiere pleno control sobre el cliente. Desde su interfaz, el servidor puede enviar distintos tipos de órdenes que el cliente ejecutará de forma inmediata. Estas órdenes pueden incluir la ejecución remota de comandos del sistema, el inicio del keylogger para registrar todas las pulsaciones del teclado del usuario, la toma de capturas de pantalla, o incluso la descarga de archivos almacenados en el equipo víctima. Todas estas acciones se llevan a cabo sin intervención del usuario del equipo cliente, y sus resultados son transmitidos, también cifrados, de vuelta al servidor.

Toda la información recopilada mediante estas acciones es almacenada de forma local en el servidor, en un directorio previamente configurado para tal fin. Esto incluye logs de comandos ejecutados, registros de teclado, archivos transferidos y capturas de pantalla. Gracias a esta estructura, el servidor puede llevar un registro detallado de toda la actividad realizada sobre cada uno de los clientes conectados.

En resumen, el flujo completo del sistema comienza con una conexión TCP iniciada por el cliente, seguida de un intercambio de claves para establecer una conexión segura. Posteriormente, se desarrolla una comunicación cifrada y continua que permite al servidor ejecutar distintas acciones sobre el cliente, cuyos resultados son almacenados para su posterior análisis. Este flujo está diseñado para garantizar tanto la eficacia del sistema como la seguridad de las comunicaciones, y permite mantener el control total sobre los clientes de forma remota, rápida y discreta.

3. OBJETIVOS CONCRETOS Y METODOLOGÍA DE TRABAJO

El presente capítulo establece la hoja de ruta técnica y metodológica seguida para el desarrollo del proyecto. Partiendo de una necesidad concreta en el ámbito de la ciberseguridad ofensiva la comprensión profunda del funcionamiento de troyanos modernos en entornos controlados se definen los objetivos operativos y el enfoque metodológico adoptado para alcanzar dicha meta.

En la actualidad, las herramientas tipo RAT (Remote Access Trojan) representan una de las amenazas más comunes y versátiles empleadas por actores maliciosos para comprometer sistemas de forma remota, recolectar información y mantener persistencia en la infraestructura víctima. Sin embargo, estas mismas capacidades, cuando se estudian desde una perspectiva ética y académica, constituyen un recurso valioso para la **formación técnica de analistas de seguridad**, profesionales de pruebas de penetración y defensores de infraestructuras críticas.

En este contexto, el proyecto plantea el desarrollo de un **troyano modular orientado a la investigación y la formación**, con un enfoque de diseño seguro, controlado y documentado. La herramienta desarrollada emula funcionalidades reales, pero su diseño garantiza que no represente una amenaza en sí misma: no se replica, no borra datos y no tiene fines destructivos. Su modularidad permite simular múltiples fases de un ataque, desde la intrusión inicial hasta la obtención de control remoto y extracción de información simulada.

El planteamiento de objetivos específicos permite descomponer el desarrollo en etapas claramente delimitadas, lo que facilita su implementación, validación y documentación. Del mismo modo, la metodología adoptada combina buenas prácticas del desarrollo de software con principios de análisis de amenazas y pruebas de concepto controladas, propias del entorno profesional de la ciberseguridad.

Este enfoque integral no solo proporciona un producto funcional, sino que genera un marco de referencia replicable para futuros desarrollos educativos o de investigación en análisis de malware, comunicación cifrada y técnicas de evasión. Así, este capítulo conecta directamente el estudio previo del dominio (capítulos anteriores) con la contribución práctica presentada en los apartados posteriores del trabajo.

3.1. OBJETIVO GENERAL

El objetivo general de este Trabajo Fin de Máster es **diseñar, desarrollar, implementar y validar un troyano modular con capacidades de acceso remoto seguro**, concebido como una herramienta técnica de análisis, evaluación y formación en ciberseguridad ofensiva, dentro de entornos controlados y éticamente justificados.

Esta herramienta no busca únicamente emular el comportamiento de un RAT (Remote Access Trojan) tradicional, sino **reproducir con fidelidad los mecanismos técnicos utilizados en amenazas reales**, incluyendo la recolección de datos en el sistema víctima, la ejecución remota de comandos arbitrarios y la transferencia encubierta de archivos. Todo ello operando bajo una arquitectura Cliente-Servidor robusta, utilizando canales de comunicación cifrados mediante técnicas de criptografía híbrida (RSA + AES) y mecanismos de evasión orientados a simular una amenaza avanzada y persistente.

El trabajo se enmarca en una **necesidad real del ámbito académico y profesional**: disponer de herramientas didácticas que permitan entender a fondo el funcionamiento interno del malware moderno, sin recurrir a muestras reales que representen un riesgo activo. De este modo, el proyecto se convierte en una **plataforma práctica para el estudio del ciclo de vida de una amenaza**, desde su implantación inicial hasta la persistencia, recolección y exfiltración de datos, en condiciones de laboratorio controladas.

Desde el punto de vista técnico, el proyecto también busca validar la eficacia de prácticas clave del desarrollo seguro, como:

- **La implementación correcta de cifrado punto a punto.**
- **El diseño modular orientado a pruebas específicas.**
- **La validación del comportamiento de la herramienta frente a soluciones defensivas activas**, como antivirus, sistemas EDR o firewalls.

Por su parte, desde una perspectiva ética y educativa, el objetivo contempla:

- La **documentación exhaustiva del código.**
- La inclusión de **advertencias de uso responsable.**
- La entrega de guías que ayuden a contextualizar el uso de la herramienta como **parte de ejercicios de red team, pruebas de penetración simuladas o formación de blue teams.**

En conjunto, el objetivo general del proyecto es proporcionar un recurso técnico completo que actúe como **entorno de simulación de amenazas reales** y, al mismo tiempo, como **referente metodológico** sobre cómo se pueden desarrollar herramientas ofensivas seguras, legales y responsables. Se trata, por tanto, de una propuesta que busca equilibrar el rigor académico, la utilidad profesional y la integridad ética, contribuyendo a formar profesionales más preparados, conscientes y capacitados frente a amenazas complejas.

3.2. OBJETIVOS ESPECÍFICOS

Para alcanzar el objetivo general propuesto —el diseño e implementación de un troyano modular con funcionalidades avanzadas de acceso remoto en entornos controlados— se han definido una serie de **objetivos específicos** que permiten desglosar el proyecto en fases operativas claras. Estos objetivos están alineados con las competencias técnicas que se esperan adquirir y demuestran la aplicabilidad práctica del trabajo en entornos de auditoría, formación y simulación de ciberamenazas.

Los objetivos específicos son los siguientes:

1. Analizar el funcionamiento y estructura técnica de troyanos y RATs reales

Se estudian en profundidad las características técnicas más comunes en herramientas maliciosas modernas, tanto de código abierto como de muestras reales analizadas en entornos de laboratorio. Este análisis permite identificar patrones de diseño, funciones recurrentes (keylogger, file manager, shell remota), vectores de infección y mecanismos de evasión y persistencia. También se revisan aspectos legales y éticos del uso de estas tecnologías con fines formativos.

2. Diseñar una arquitectura modular orientada a control, escalabilidad y pruebas

Se define una arquitectura basada en un modelo **Cliente-Servidor**, con un diseño modular que permita añadir, actualizar o desactivar funcionalidades de forma sencilla. Cada módulo (captura de teclado, shell remota, transferencia de archivos, etc.) debe poder operar de forma autónoma y ser configurable, permitiendo así ejecutar pruebas específicas o escenarios

personalizados. Este diseño también facilita el mantenimiento y la adaptación de la herramienta a nuevas funcionalidades en el futuro.

3. Desarrollar funcionalidades fundamentales de un troyano avanzado

Se implementan los módulos clave del troyano, asegurando que cada uno de ellos reproduzca de manera fiel una función típica de un RAT real:

- **Keylogger:** Captura en segundo plano de pulsaciones del teclado.
 - **File manager:** Envío y recepción de archivos desde y hacia el servidor.
 - **Remote shell:** Ejecución remota de comandos del sistema operativo desde el servidor hacia el cliente.
El código se desarrolla en Python por su rapidez de prototipado y amplio ecosistema de bibliotecas de red y cifrado.
-

4. Implementar un sistema de comunicación segura mediante cifrado híbrido

Se establece un protocolo de intercambio de claves RSA-AES para asegurar la confidencialidad de las comunicaciones entre cliente y servidor:

- El servidor genera un par de claves RSA.
- El cliente genera una clave AES por sesión y la cifra con la clave pública del servidor.
- Todos los mensajes se cifran simétricamente con AES-CBC, asegurando confidencialidad y eficiencia.

Este sistema impide que una interceptación del tráfico permita recuperar los datos transmitidos, cumpliendo con principios de seguridad como el *confidentiality-by-design*.

5. Integrar un sistema de túnel TCP (como Ngrok) para permitir conexiones externas

Para simular escenarios reales de conexión desde redes externas, se utiliza **Ngrok** como solución de redireccionamiento de puertos a través de NAT/firewalls. Esta herramienta permite exponer el puerto del servidor a través de una URL pública temporal, sin necesidad de configurar manualmente el router o el firewall. Su integración reproduce las técnicas que muchos RATs reales emplean para evadir restricciones de red y garantizar persistencia en entornos con configuraciones cerradas.

6. Verificar el correcto funcionamiento técnico de cada módulo

Se diseñan pruebas unitarias e integradas que validan el comportamiento esperado de cada módulo del troyano. Estas pruebas incluyen:

- Comprobación de envío y recepción de archivos sin pérdida de integridad.
 - Validación del cifrado efectivo de las comunicaciones mediante herramientas como Wireshark.
 - Confirmación de la ejecución remota de comandos sin errores.
 - Simulación de sesiones en entornos virtuales para detectar fallos o comportamientos anómalos.
-

7. Evaluar la capacidad de evasión y detección del troyano frente a defensas activas

Se ejecutan pruebas en presencia de antivirus, EDR (Endpoint Detection & Response) y otras herramientas defensivas, para evaluar si el troyano puede ser detectado. También se analiza si el tráfico cifrado puede ser identificado o clasificado como sospechoso por soluciones de análisis de red. Estas pruebas permiten establecer un perfil de “riesgo controlado” para cada módulo.

8. Documentar técnica, funcional y éticamente el sistema

Se redacta una documentación completa del proyecto que incluye:

- Especificaciones técnicas de la arquitectura y cada módulo.
- Manual de usuario para reproducir el entorno de pruebas de forma segura.
- Declaración de limitaciones, advertencias de uso y marco legal.
- Guía para su aplicación en prácticas formativas o simulacros de respuesta a incidentes.

3.3. METODOLOGÍA DEL TRABAJO

La metodología aplicada en este Trabajo Fin de Máster se basa en una combinación de enfoques del **desarrollo seguro de software**, principios de **ingeniería inversa** y prácticas

profesionales empleadas en **pruebas de penetración y análisis de malware**. Se ha seguido una estrategia estructurada y cíclica que integra tanto el análisis previo como el diseño, la implementación técnica, la validación funcional y la reflexión ética sobre el uso del sistema desarrollado.

El enfoque metodológico adoptado se articula en las siguientes fases principales:

1. Revisión bibliográfica y técnica del dominio

Se realizó una revisión del estado del arte en materia de:

- Troyanos y herramientas RAT reales, analizando su estructura, funcionalidades y técnicas de persistencia.
- Protocolos de comunicación y cifrado empleados por malware avanzado.
- Herramientas y entornos utilizados para pruebas de penetración.
- Aspectos legales y éticos relacionados con el desarrollo de software potencialmente malicioso en entornos académicos.

Esta revisión sirvió de base para delimitar el alcance funcional del proyecto y establecer una hoja de ruta realista, alineada con los objetivos del Máster y la legislación vigente.

2. Análisis de requisitos

Se definieron **requisitos funcionales y no funcionales** del sistema. Entre los funcionales destacan:

- Captura de teclado (keylogging).
- Transferencia de archivos bidireccional.
- Ejecución remota de comandos desde servidor.
- Comunicación cifrada entre cliente y servidor.

Entre los no funcionales:

- Modularidad y configuración flexible del troyano.
- Seguridad en la ejecución (solo en entornos virtualizados).
- Cifrado robusto y documentación clara del uso.

Este análisis se realizó con enfoque *threat-driven*, evaluando las capacidades mínimas necesarias para simular una amenaza real sin implicar daño alguno.

3. Diseño de arquitectura modular

El diseño técnico se basó en un modelo **Cliente-Servidor modular**. Se optó por una arquitectura desacoplada donde cada módulo funcional se implementa como componente independiente, fácilmente integrable al sistema general. Esta decisión favorece:

- La escalabilidad futura del sistema.
- La posibilidad de aislar y probar cada módulo por separado.
- El cumplimiento del principio de *separation of concerns*, mejorando la mantenibilidad del código.

El servidor actúa como centro de control, gestionando múltiples conexiones de clientes y enviando instrucciones cifradas a través de un canal seguro.

4. Desarrollo seguro del prototipo

La implementación se llevó a cabo utilizando **Python**, por su extensibilidad, legibilidad y disponibilidad de bibliotecas criptográficas y de red. Los principales paquetes empleados fueron:

- `cryptography` para gestión de claves RSA y cifrado AES.
- `socket` para la conexión TCP entre cliente y servidor.
- `pynput` para la captura de teclas.
- `os` y `subprocess` para ejecutar comandos del sistema desde el servidor.

Durante el desarrollo se aplicaron buenas prácticas como:

- Validación de entradas.
 - Manejo de excepciones.
 - Separación de lógica de presentación y control.
 - Restricción de permisos y operaciones peligrosas.
-

5. Implementación del canal cifrado

Una de las fases críticas del trabajo fue la implementación del canal seguro:

- Se generó un par de claves RSA en el servidor.
- El cliente, al iniciar sesión, genera una clave AES de sesión.
- Esta clave AES se cifra con la clave pública del servidor y se envía por el canal.
- Todo el tráfico posterior se cifra simétricamente con AES en modo CBC.

Este modelo de **cifrado híbrido** emula sistemas reales como TLS, garantizando **confidencialidad, integridad y autenticación básica**.

6. Integración de túnel de red (Ngrok)

Para simular escenarios realistas donde el atacante se conecta desde una red externa, se utilizó **Ngrok** como sistema de tunelización TCP. Esta herramienta permitió exponer el puerto del servidor a través de Internet sin configurar redirecciones de puertos, replicando condiciones comunes de ataques RAT que utilizan técnicas de *reverse shell* y *cloud relay*.

7. Validación funcional y de seguridad

Se diseñó una batería de pruebas para verificar:

- El correcto funcionamiento de cada módulo.
- La estabilidad de las sesiones.
- La confidencialidad de las comunicaciones (verificada con Wireshark).
- La resistencia a detección por parte de antivirus comerciales y sistemas EDR.

Además, se probó la herramienta en entornos virtualizados (VirtualBox y VMware) para garantizar que el comportamiento no se propagara más allá del entorno de prueba.

4. DESARROLLO ESPECÍFICO DE LA CONTRIBUCIÓN

En este bloque se presenta de forma detallada la principal contribución de este Trabajo Fin de Máster: el desarrollo de una herramienta software que simula el funcionamiento de un malware tipo Remote Access Trojan (RAT), que hemos denominado Infiltrator. La herramienta ha sido concebida como una plataforma de pruebas para estudiar técnicas de acceso remoto, persistencia, recolección de información y evasión de detección, dentro de un entorno controlado y con fines exclusivamente académicos.

El desarrollo de esta herramienta no persigue su uso malicioso, sino que pretende ofrecer un enfoque técnico y práctico que permita comprender cómo operan este tipo de amenazas en escenarios reales, aportando valor al ámbito de la ciberseguridad desde una perspectiva defensiva. La aplicación se ha diseñado en lenguaje C# bajo el framework .NET y consta de dos módulos principales: un servidor de control y un cliente agente que simula el equipo comprometido.

Este capítulo se estructura en tres secciones. En primer lugar, se identifican los requisitos funcionales y no funcionales del sistema, así como el contexto y el problema que se busca abordar. A continuación, se describe en detalle la herramienta desarrollada, desde el proceso de desarrollo hasta su arquitectura interna, ilustrada mediante diagramas y capturas de pantalla. Finalmente, se presenta una evaluación cualitativa de la herramienta, enfocada en su usabilidad, estabilidad y aplicabilidad como recurso formativo o de análisis en el estudio de amenazas tipo APT.

4.1 TIPO 2. DESARROLLO DE SOFTWARE

4.1.1. Identificación de requisitos

Comenzamos este proyecto con la idea de que actualmente, muchos laboratorios académicos y entornos formativos en ciberseguridad carecen de recursos realistas y funcionales para analizar cómo se comportan los malware tipo RAT en escenarios simulados. El uso de muestras reales en entornos de formación conlleva riesgos legales, éticos y operacionales. Frente a esto, surge la necesidad de diseñar una herramienta propia y controlada, con funcionalidades equiparables a las de un malware real, pero desarrollada con fines educativos e investigativos.

Contexto de uso

Este desarrollo se enmarca en un contexto académico, dentro de un entorno aislado y seguro, sin conexión a redes productivas ni a Internet. El sistema puede utilizarse en universidades, centros de formación técnica o laboratorios de análisis de malware, como recurso para la formación de especialistas en ciberseguridad y para la experimentación con mecanismos de detección y respuesta ante amenazas.

Participación de expertos

Durante la definición de los requisitos y el diseño funcional de la herramienta, se ha contado con la supervisión del director del TFG, con experiencia en el ámbito de la ingeniería informática y los sistemas de mando y control. Asimismo, se han revisado informes técnicos y documentación de análisis forense de APTs reales, publicados por empresas especializadas como FireEye, Kaspersky, MITRE ATT&CK, entre otras, con el objetivo de replicar comportamientos legítimos de malware y estructurar una arquitectura modular y extensible.

Requisitos funcionales

Los requisitos funcionales que hemos identificado para la herramienta son los siguientes:

- **RF01:** El sistema debe permitir establecer una conexión TCP entre un cliente y el servidor.
- **RF02:** El cliente debe enviar al servidor información del sistema (IP, nombre del equipo, usuario, versión SO, RAM, CPU, GPU, etc.).
- **RF03:** El servidor debe poder enviar comandos al cliente y recibir la salida en tiempo real.
- **RF04:** El cliente debe ser capaz de capturar entradas del teclado (keylogger).
- **RF05:** El cliente y el servidor deben permitir la transferencia de archivos bilateralmente.
- **RF06:** El sistema debe implementar un cifrado híbrido (RSA/AES) para proteger las comunicaciones.

Requisitos no funcionales

- **RNF01:** La herramienta debe ejecutarse únicamente en entornos Windows.

- **RNF02:** La interfaz del servidor debe ser gráfica y permitir la gestión de múltiples clientes.
- **RNF03:** El sistema debe evitar técnicas invasivas o destructivas, garantizando la seguridad del entorno de prueba.
- **RNF04:** La arquitectura debe estar diseñada para facilitar la futura integración de nuevos módulos.

Este conjunto de requisitos ha guiado el proceso de diseño y desarrollo de *Infiltrator*, garantizando la viabilidad, aplicabilidad y seguridad del experimento en el marco de este trabajo.

4.1.2. Descripción detallada de la herramienta software desarrollada

La herramienta desarrollada, denominada Infiltrator Project, es una implementación funcional de un malware RAT, diseñada para operar en entornos controlados con fines académicos y de investigación. El sistema se compone de dos módulos principales: el cliente (implantado en la máquina objetivo) y el servidor de control (C2), encargado de la gestión de sesiones y el envío de instrucciones.

Fases del desarrollo

Dividimos el desarrollo en las siguientes iteraciones o etapas:

1. Análisis de requisitos y diseño del sistema

- Estudio de técnicas utilizadas en amenazas reales.
- Diseño de una arquitectura cliente-servidor basada en sockets TCP.
- Definición de protocolos de comunicación personalizados.

2. Desarrollo del servidor

- Implementado en **C#** con interfaz gráfica en **Windows Forms**.
- Módulo de escucha (listener) para gestionar múltiples conexiones TCP.
- Registro e identificación de clientes mediante información del sistema.
- Gestión de sesiones activas mediante DataGridView.
- Soporte para envío de comandos y recepción de resultados.

3. Desarrollo del cliente

- Aplicación en **C#** ejecutada en segundo plano y sin interfaz.
- Envío de datos del sistema al servidor.

- Captura de teclado y ventanas activas.
- Persistencia en el sistema (simulada en entorno controlado).
- Recepción de comandos, ejecución local y respuesta al servidor

4. Implementación de seguridad en las comunicaciones y funciones

- Cifrado híbrido AES/RSA para proteger la comunicación cliente-servidor.
- Intercambio de clave pública del servidor durante la conexión inicial.
- Cifrado simétrico de los mensajes con clave AES compartida.

5. Pruebas y validación

- Se desplegó el sistema en máquinas virtuales aisladas.
- Se realizaron pruebas de conexión, ejecución de comandos y captura de datos.
- Se verificó la robustez del sistema ante múltiples sesiones concurrentes.

A continuación, explicaremos cada etapa de desarrollo:

Análisis de requisitos y diseño del sistema

El desarrollo de la herramienta presentada en este Trabajo de Fin de Grado parte de un análisis riguroso del comportamiento y las técnicas utilizadas en amenazas persistentes avanzadas (APT), con el objetivo de simular, en un entorno controlado, las capacidades básicas de un malware de tipo RAT (Remote Access Trojan). A continuación, se describen las bases sobre las que se ha diseñado la arquitectura del sistema y su protocolo de funcionamiento.

Para garantizar un enfoque realista, se realizó un estudio preliminar de distintas familias de malware modernas, como Gh0st RAT, njRAT o DarkComet. Este análisis permitió identificar un conjunto de técnicas comunes empleadas por dichas amenazas, entre las que destacan:

- **Uso de sockets TCP para establecer canales de control encubiertos.**
- **Cifrado de las comunicaciones** mediante algoritmos simétricos (AES) o híbridos (RSA + AES).
- **Persistencia en el sistema comprometido** y ejecución automática tras reinicio (no implementado en esta fase del proyecto).
- **Obtención de información sensible del sistema** (nombre de usuario, SO, GPU, etc.).
- **Interfaz de operador para control remoto de múltiples víctimas.**

Estos elementos han sido considerados como base para el diseño de la herramienta desarrollada, con el fin de aproximarse al funcionamiento de una amenaza real dentro de un entorno seguro de pruebas.

La arquitectura del sistema se basa en un modelo clásico cliente-servidor, donde el servidor actúa como centro de mando y control (C2), y los clientes representan sistemas comprometidos que se conectan activamente al servidor. Para la implementación de esta arquitectura se ha empleado el uso de sockets TCP, aprovechando su fiabilidad y simplicidad para la transmisión de datos entre los extremos.

El servidor escucha de forma asíncrona en un puerto definido (por defecto, 443) y acepta conexiones entrantes de múltiples clientes. Cada cliente mantiene una conexión persistente mediante un `TcpClient`, desde donde se pueden recibir y ejecutar órdenes. Este enfoque facilita la gestión simultánea de múltiples sesiones activas sin bloqueos ni pérdida de rendimiento.

Con el fin de dotar al sistema de un canal de comunicación seguro y eficiente, se ha diseñado un protocolo de comunicación binario personalizado. Este protocolo está compuesto por una cabecera fija que define el tipo y longitud del mensaje, seguida del cuerpo del mensaje cifrado.

Los mensajes intercambiados entre cliente y servidor se estructuran de la siguiente manera:

[TipoMensaje (1 byte)] [Tamaño (4 bytes)] [Contenido cifrado (N bytes)]

Donde:

- **TipoMensaje** indica la operación a realizar (información del sistema, ejecución de comando, envío de archivo, etc.).
- **Tamaño** especifica el tamaño del contenido cifrado.
- **Contenido cifrado** contiene los datos sensibles, protegidos mediante cifrado AES (clave intercambiada mediante RSA).

Esta estructura facilita la interpretación y el procesamiento automatizado de comandos en ambos extremos del canal. Además, al encapsular toda la información en mensajes cifrados, se garantiza la confidencialidad de la comunicación, incluso en entornos inseguros o monitorizados.

Desarrollo del servidor

El servidor de mando y control desarrollado en el marco de este proyecto ha sido implementado en lenguaje C#, aprovechando el entorno de desarrollo .NET y la biblioteca Windows Forms para la creación de una interfaz gráfica de usuario funcional y accesible. El objetivo principal de este componente es permitir al operador gestionar múltiples clientes comprometidos desde una única consola centralizada, facilitando tanto la supervisión como la ejecución remota de acciones sobre los sistemas conectados.

En el núcleo del servidor se encuentra un módulo de escucha (listener), basado en la clase TcpListener, que se encarga de abrir un puerto determinado —configurable, siendo el 443 el valor predeterminado— y aceptar conexiones entrantes de forma asíncrona. Esta aproximación permite mantener una elevada concurrencia sin comprometer la capacidad de respuesta del sistema. Cada cliente que establece una conexión es gestionado en un hilo independiente o mediante tareas asíncronas, lo que evita bloqueos en la interfaz y garantiza una experiencia fluida al operador.

Al recibir una nueva conexión, el servidor solicita automáticamente al cliente que proporcione información básica del sistema, como la dirección IP, el puerto local, el nombre del equipo, el nombre de usuario, la versión del sistema operativo y las características de la GPU. Esta información es enviada cifrada desde el cliente y, una vez descifrada, se registra en la interfaz principal del servidor dentro de un DataGridView. Esta tabla permite al operador identificar rápidamente a cada cliente, consultar su estado, y realizar operaciones individualizadas o masivas sobre ellos.

El servidor también proporciona un sistema para el envío de comandos personalizados a los clientes conectados. Para ello, el operador puede seleccionar uno o varios sistemas desde el DataGridView y emitir instrucciones que son codificadas mediante el protocolo definido y transmitidas por el canal TCP. Los resultados de la ejecución se reciben de forma cifrada, se descifran en el servidor y se presentan al operador a través de cuadros de texto o elementos visuales adaptados al tipo de respuesta recibida (por ejemplo, texto plano, captura de pantalla, etc.).

Esta arquitectura modular y extensible ha sido diseñada para facilitar la incorporación de futuras funcionalidades, como transferencia de archivos, ejecución de scripts, o control

remoto más avanzado, manteniendo siempre un equilibrio entre simplicidad de uso y robustez del sistema.

Desarrollo del cliente

El cliente desarrollado para este proyecto consiste en una aplicación escrita en C# que opera de forma completamente oculta al usuario, sin mostrar interfaz gráfica y ejecutándose en segundo plano. Su propósito es establecer una conexión persistente con el servidor de mando y control, proporcionar información del sistema objetivo y ejecutar comandos recibidos de manera remota. Todo el funcionamiento del cliente ha sido diseñado bajo un enfoque modular y orientado a eventos, con especial atención a la evasión de detección, el rendimiento y la fiabilidad en un entorno simulado y controlado.

Una vez iniciado, el cliente intenta establecer conexión con la dirección IP y el puerto TCP especificados en su configuración. Si la conexión tiene éxito, se procede a un intercambio inicial cifrado en el que el cliente recopila y envía datos relevantes del sistema: dirección IP, nombre del equipo, nombre del usuario actual, versión del sistema operativo, información de hardware como la GPU, y el puerto local utilizado. Esta información permite al servidor identificar al cliente de forma única y registrar sus características para posteriores interacciones.

Durante su ejecución, el cliente mantiene una escucha pasiva de instrucciones desde el servidor. Al recibir un comando válido, se interpreta y se ejecuta en el contexto del sistema local. Los resultados de dicha ejecución, ya sean salidas de texto, capturas de pantalla u otros tipos de respuesta, son recogidos, cifrados y enviados de vuelta al servidor. Esta comunicación bidireccional se realiza sobre sockets TCP utilizando un protocolo propio y ligero que encapsula tanto los comandos como sus resultados de manera segura.

Entre las funcionalidades implementadas, destaca la captura periódica del estado del sistema, que incluye tanto las teclas pulsadas como las ventanas activas en el sistema operativo. Estos datos son almacenados temporalmente y transmitidos al servidor en intervalos definidos, simulando el comportamiento de una amenaza persistente avanzada. Para el registro de pulsaciones se emplea un hook global del teclado, mientras que para la detección de ventanas activas se consulta el identificador de la ventana en primer plano.

Asimismo, se ha implementado una técnica de persistencia simulada para asegurar la permanencia del cliente tras reinicios del sistema, aunque esta funcionalidad se ha desarrollado y probado únicamente en un entorno cerrado y controlado, sin afectar sistemas reales. La persistencia se simula mediante el registro de claves en ubicaciones del sistema como el "Run" del registro de Windows y copias del ejecutable en rutas estándar de inicio automático.

En conjunto, el cliente ha sido diseñado como una entidad autónoma, resiliente y capaz de mantener una comunicación fluida con el servidor, proporcionando las capacidades básicas de una herramienta de administración remota (RAT), enmarcada en un proyecto académico de investigación y desarrollo de software con fines exclusivamente formativos y experimentales.

Implementación de seguridad en las comunicaciones y funciones

Con el objetivo de proteger la confidencialidad e integridad de las comunicaciones entre el servidor y los clientes, se ha implementado un esquema de cifrado híbrido basado en la combinación de RSA (cifrado asimétrico) y AES (cifrado simétrico). Este enfoque permite aprovechar la eficiencia del cifrado simétrico para la transmisión de datos, al tiempo que garantiza un intercambio seguro de claves mediante criptografía asimétrica, minimizando el riesgo de interceptación o manipulación de la información durante su tránsito.

Durante la fase de conexión inicial, el servidor envía su clave pública RSA al cliente. Esta clave se utiliza exclusivamente para cifrar una clave AES generada aleatoriamente por el cliente, la cual será empleada como clave de sesión para todas las comunicaciones posteriores. Una vez cifrada, la clave AES se transmite al servidor, que puede descifrarla utilizando su clave privada RSA. De este modo, ambas partes comparten una clave simétrica de forma segura, sin necesidad de transmitirla en texto plano.

A partir de ese momento, todos los mensajes intercambiados entre el cliente y el servidor incluyendo los comandos, las respuestas, los datos del sistema y cualquier información sensible se cifran utilizando el algoritmo AES en modo CBC o similar, con la clave de sesión compartida. Este esquema asegura tanto la confidencialidad como la integridad de los mensajes, dificultando su análisis incluso en caso de interceptación del tráfico.

Además del cifrado de las comunicaciones, se han adoptado otras medidas de seguridad en la implementación de funciones críticas. Por ejemplo, los comandos se encapsulan en estructuras definidas que incluyen validación de integridad y longitud, y el canal de comunicación TCP se monitoriza para evitar desbordamientos o peticiones mal formadas. Asimismo, los datos sensibles enviados al servidor como pulsaciones de teclado o nombres de procesos activos son codificados y cifrados antes de su transmisión, impidiendo su recuperación mediante análisis de memoria o tráfico de red.

Este modelo de seguridad híbrida proporciona un nivel de protección suficiente para garantizar la privacidad de las operaciones dentro del entorno experimental del proyecto, emulando mecanismos utilizados por herramientas reales de administración remota o amenazas avanzadas persistentes (APT), siempre dentro de un marco ético y controlado.

Pruebas y validación

Con el objetivo de comprobar el correcto funcionamiento de la herramienta desarrollada, se llevó a cabo un proceso de pruebas y validación en un entorno controlado, utilizando máquinas virtuales aisladas del entorno de red real. Este enfoque permitió simular escenarios de ejecución sin comprometer la seguridad del sistema anfitrión ni incumplir principios éticos, dado el carácter sensible del software desarrollado.

En primer lugar, se desplegó el servidor en una máquina virtual con sistema operativo Windows, y posteriormente se lanzaron múltiples instancias del cliente en otras máquinas virtuales configuradas con diferentes versiones del mismo sistema operativo. Estas instancias fueron ejecutadas en segundo plano, replicando el comportamiento previsto en un escenario real de uso del software.

Las pruebas funcionales incluyeron la verificación de la conexión inicial entre cliente y servidor, el intercambio correcto de claves para establecer una sesión segura mediante el cifrado híbrido AES/RSA, y el registro de la información del sistema por parte del servidor. Asimismo, se validó la correcta recepción y ejecución de comandos desde el servidor, así como la capacidad del cliente para enviar de vuelta la salida correspondiente de forma cifrada y estructurada.

Adicionalmente, se llevaron a cabo pruebas específicas orientadas a validar la fiabilidad del sistema en condiciones de uso intensivo. Se simulaban múltiples conexiones concurrentes

para comprobar que el servidor era capaz de gestionar correctamente varias sesiones activas, actualizando en tiempo real el estado de cada cliente conectado mediante la interfaz gráfica y el control DataGridView. Durante estas pruebas se observaron tiempos de respuesta aceptables, sin pérdidas de conexión ni errores de sincronización entre mensajes.

Por último, se realizaron pruebas de captura de datos sensibles como pulsaciones de teclado y ventanas activas, verificando su envío correcto al servidor y su registro adecuado en los módulos correspondientes. La validación incluyó la revisión manual de los datos recibidos, comprobando que se mantenía la integridad de los mismos durante el tránsito.

Este conjunto de pruebas permitió confirmar la estabilidad, funcionalidad y capacidad del sistema desarrollado para operar de acuerdo con los objetivos planteados, asegurando que el comportamiento del malware simulado se ajusta a lo esperado dentro de los límites éticos y técnicos definidos para este Trabajo de Fin de Grado.

4.1.3. Evaluación

La evaluación del sistema desarrollado se centró en dos aspectos clave: su capacidad para cumplir con los objetivos funcionales definidos y su usabilidad desde el punto de vista del operador del servidor. Dado el carácter sensible del software (una herramienta simulada con funcionalidades propias de un malware tipo APT), todas las pruebas y validaciones se realizaron en un entorno aislado y controlado, con fines exclusivamente académicos y experimentales.

En primer lugar, se evaluó la aplicabilidad de la herramienta para simular con fidelidad el comportamiento de un malware avanzado. Durante las pruebas, el sistema demostró una correcta gestión de múltiples conexiones cliente, una comunicación segura mediante cifrado híbrido y la ejecución eficaz de funcionalidades típicas como el envío de información del sistema, la ejecución remota de comandos y la captura de datos sensibles. Esta funcionalidad refleja de manera realista las capacidades observadas en amenazas reales, lo que valida el valor del sistema como herramienta de experimentación y análisis.

Desde el punto de vista de la usabilidad, se valoró positivamente la interfaz gráfica del servidor, desarrollada en Windows Forms. Gracias al uso de controles como DataGridView, el operador pudo gestionar y visualizar de manera intuitiva las sesiones activas, el estado de los clientes y los resultados de los comandos ejecutados. Además, el diseño modular del

servidor permite su extensión con nuevas funcionalidades, lo cual lo hace adaptable a diferentes escenarios de simulación.

En cuanto al rendimiento, se observó una respuesta estable bajo la gestión simultánea de múltiples sesiones. No se detectaron pérdidas de datos ni cuelgues durante las sesiones prolongadas, lo cual indica una implementación robusta en lo que respecta a la gestión de sockets y al procesamiento asincrónico.

Finalmente, cabe destacar que, si bien no se llevó a cabo una evaluación formal con usuarios externos o expertos en ciberseguridad, los resultados obtenidos durante las pruebas internas son indicativos del correcto funcionamiento y aplicabilidad del software para su propósito experimental. La herramienta desarrollada puede considerarse válida para la simulación, análisis y comprensión de amenazas cibernéticas en contextos académicos o de investigación, siempre dentro de un marco ético y legal.